Smart Dental Clinic – Backend Documentation (Node.js + MongoDB)

This document explains the backend part of the Smart Dental Clinic system in a clear, detailed, and discussion-ready way. It is based on the provided code files (server.js, Patient.js, Record.js) and describes the architecture, database design, API endpoints, and how the backend connects with the AI Streamlit application.

1) System Architecture Overview

The project follows a clean, modular architecture where AI inference and user interaction are separated from data persistence:

- Streamlit App (Python): UI for uploading images + running YOLO detection and multi-label classification (ResNet).
- Node.js API (Express): Receives patient information and AI results, validates them, and stores them in MongoDB.
- MongoDB Database: Stores Patients and their medical Records (history + AI results + optional doctor notes).

High-level data flow:

```
User uploads image in Streamlit
    -> Streamlit runs YOLO + ResNet (AI inference)
    -> Streamlit POST /api/patients (create patient)
    <- returns patientId
    -> Streamlit POST /api/records  (save AI diagnosis record linked to patientId)
    -> MongoDB stores patient + records
```

2) Backend Technology Stack

Runtime	Node.js
Web Framework	Express.js
Database	MongoDB
ODM	Mongoose
Cross-Origin Support	cors middleware
Data Format	JSON over HTTP REST API
Default Port	5000
Base API Prefix	/api

3) Project Structure (Backend)

A typical folder structure for this backend is:

```
SmartDentalBackend/
  ├─ server.js
  ├─ models/
  │   ├─ Patient.js
  │   └─ Record.js
  ├─ package.json
  └─ node_modules/
```

The Streamlit application points to the backend using:

```
API_URL = "http://localhost:5000/api"
```

4) Database Design (MongoDB + Mongoose)

We use two collections: Patients and Records. A Record references a Patient using MongoDB ObjectId.

4.1) Patient Model (models/Patient.js)

Purpose: Store the basic demographics and contact information of a patient.

- name (String, required): Patient full name
- age (Number): Patient age
- gender (String): Male/Female
- phone (String): Phone number
- history (String): medical history / notes collected from the UI
- createdAt (Date, default now): created automatically by the database

Implementation (excerpt):

```
const mongoose = require('mongoose');

const PatientSchema = new mongoose.Schema({
  name: { type: String, required: true },
  age: Number,
  gender: String,
  phone: String,

  // ✅ مهم: Streamlit بيبعته باسم history
  history: { type: String, default: "" },

  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Patient', PatientSchema);
```

4.2) Record Model (models/Record.js)

Purpose: Store each diagnosis event (one uploaded image) and connect it to the patient.

- patientId (ObjectId, required): Link to Patient collection
- imagePath (String): path or filename for the stored image (future improvement: save actual file URL)
- aiResult (Mixed): A flexible JSON object storing detected diseases, confidence scores, triage decision, etc.
- doctorNotes (String): optional notes from the doctor or from the patient history field
- createdAt (Date, default now): timestamp of the diagnosis record

Implementation (excerpt):

```
const mongoose = require('mongoose');

const RecordSchema = new mongoose.Schema({
  patientId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Patient',
    required: true
  },

  imagePath: String,

  // ✅ خلي aiResult مرن يخزن أي JSON (YOLO detections + thresholds + classification + summary...)
  aiResult: mongoose.Schema.Types.Mixed,

  doctorNotes: String,
  visitDate: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Record', RecordSchema);
```

4.3) Why this design works well

- One patient can have many records → supports follow-ups and longitudinal tracking.
- Records can store flexible AI output via Mongoose Mixed type without breaking schema when you add new AI fields later.
- The reference (patientId) allows efficient querying per patient and future "populate" joins if needed.

5) REST API Endpoints

All endpoints are under: http://localhost:5000/api

5.1) GET /health

Used for checking if the backend is alive.

Response	{"status":"Backend is running ☑"}
Status Code	200 OK

5.2) POST /patients

Creates a new patient and returns the created patient object including the MongoDB _id.

Request JSON example:

```
{
  "name": "Ahmed Ali",
  "age": 30,
  "gender": "Male",
  "phone": "01000000000",
  "history": "Diabetes, smoker"
}
```

Key behavior behind the scenes:

- Validates required field 'name' (Mongoose).

- Uses async/await to save the patient to MongoDB.
- Returns status 201 and the new patient JSON if successful.
- Returns status 500 if database saving fails (with error message).

5.3) GET /patients

Returns a list of all patients.

- Useful for building a dashboard later (patient search, list, filtering).
- Current implementation returns the raw array from MongoDB.

5.4) POST /records

Creates a new diagnosis record linked to an existing patient.

Request JSON example:

```
{
  "patientId": "6560c1b1a...",
  "imagePath": "Uploaded_Image.jpg",
  "aiResult": {
    "detected_diseases": ["Data caries", "Mouth Ulcer"],
    "confidence_scores": [0.87, 0.76],
    "triage_level": "🔴 High Priority",
    "action_taken": "Urgent"
  },
  "doctorNotes": "Patient complains of pain."
}
```

Server behavior:

- Validates that patientId is provided → otherwise returns 400.
- Checks if the patient exists in MongoDB → if not found returns 404.
- Saves the record and returns 201 with the created record JSON.

5.5) GET /records/patient/:patientId

Returns all records for a specific patient, sorted by date (latest first).

- Supports patient history timeline view.
- Used to build doctor dashboard (records per patient).

5.6) Error Handling and Status Codes

200	OK (for GET requests)
201	Created (new patient/record)
400	Bad Request (missing required data like patientId)
404	Not Found (patient does not exist)
500	Server Error (database or unexpected error)

6) Behind the Scenes: What Happens Internally

Even though the code is short, a lot happens under the hood:

6.1) Express Middleware Pipeline

- cors(): Allows the Streamlit UI (or any other client) to call the API from a different origin.
- express.json(): Parses incoming JSON body and makes it available as req.body.
- Routes: match URL paths (e.g., /api/patients) and execute the corresponding handler.

6.2) Mongoose Connection + Models

- mongoose.connect(): opens a connection pool to MongoDB and reuses connections efficiently.
- Schema validation: required fields and type checks occur during save().
- ObjectId references: enable relationships between collections.

6.3) Node.js Event Loop + Async/Await

MongoDB operations are I/O bound. Node.js uses an event loop, so the server can handle multiple requests without blocking. Using async/await makes the code readable while still non-blocking.

7) Integration with the Streamlit AI App

In the Streamlit UI, the 'Save to Database' button triggers a two-step process:

1. POST /api/patients with sidebar data (name, age, gender, phone, history).
2. If successful, extract patientId from response JSON (_id).
3. POST /api/records with patientId + AI output (detected diseases and confidence) + doctor notes.

This design prevents orphan records and ensures every record is attached to a valid patient.

8) Local Setup and Run Instructions

Prerequisites: Node.js installed + MongoDB running locally.

Install dependencies:

```
npm install express mongoose cors
```

Run the server:

```
node server.js
```

Expected console messages:

```
✅ Connected to MongoDB
🚀 Server running on port 5000
```

Quick test (browser or Postman): GET http://localhost:5000/api/health

9) Recommended Improvements (Production-Grade)

- Environment variables (.env) for MongoDB URI and PORT (avoid hardcoding).
- Authentication & authorization (JWT) for doctors vs. admins.
- Input validation library (Joi/Zod) to validate requests before hitting database.
- File upload endpoint (multer) and store images in disk/cloud (S3) then store image URL in Record.

- Logging (winston/morgan), error monitoring, and rate limiting.
- Pagination for GET /patients and GET /records to scale with large data.

10) Work Division for the Team (3 AI + 3 Backend)

10.1) AI Team (3 members)

- AI-1 (Detection lead): Explain YOLO dataset, annotation format (YOLO labels), training configuration, evaluation metrics (mAP@0.5, mAP@0.5:0.95).
- AI-2 (Multi-label classification lead): Explain ResNet multi-label setup, label encoding, thresholding, and evaluation (precision/recall/F1 per label).
- AI-3 (Integration + Clinical logic): Explain how outputs are combined, triage rules, and how AI results turn into actionable recommendations.

10.2) Backend Team (3 members)

- BE-1 (API lead): Explain Express server design, endpoints, status codes, and error handling.
- BE-2 (Database lead): Explain MongoDB design, schemas (Patient/Record), relations, and queries for patient history.
- BE-3 (System integration): Explain Streamlit ↔ API connection, request/response flow, and how data is saved and later retrieved for dashboards.

11) Future Work (Planned Extensions)

11.1) NLP Chatbot for Doctors and Patients

We plan to build a medical chatbot that can answer questions based on:

- General dental knowledge (educational content for patients).
- Clinical guidelines and curated sources (for doctors).
- Patient-specific data (previous records and history) to provide personalized explanations.

Implementation idea (high level):

- RAG (Retrieval-Augmented Generation): store guidelines and FAQ in a vector database, retrieve relevant chunks, then generate a grounded answer.
- Safety layer: reject unsafe/diagnosis-final answers and always recommend doctor confirmation.
- Multi-language support (Arabic/English) to match clinic needs.

11.2) Automated Medical Report Generation

We plan to generate a structured PDF/Word report automatically after each diagnosis, including:

- Patient information + visit timestamp.
- Detected conditions with confidence scores.
- Localized findings (YOLO boxes) with image snapshots.
- Recommended next steps and triage priority.
- Doctor notes section for manual edits.
- Follow-up reminders (e.g., re-check in 2 weeks).

This report can be produced in two modes:

- Template-based (deterministic): ensures consistent clinical formatting.
- LLM-assisted narrative summary: turns structured AI outputs into a readable paragraph, with strict constraints to avoid hallucinations.

Appendix A: Backend Source Code (Reference)

server.js (full):

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const Patient = require('./models/Patient');
const Record = require('./models/Record');
const app = express();
app.use(express.json());
app.use(cors());

// ✅ MongoDB URI
const MONGO_URI = "mongodb://localhost:27017/dental_db";

mongoose.connect(MONGO_URI)
  .then(() => console.log("✅ Database Connected Successfully!"))
  .catch(err => console.error("❌ Connection Error:", err));

// ✅ Health check (اختياري لكنه مفيد للتأكد إن السيرفر شغال)
app.get('/api/health', (req, res) => {
  res.json({ status: "ok", message: "Smart Dental API is running." });
});

// ====================================================
// Patients
// ====================================================

// 1) إضافة مريض جديد
app.post('/api/patients', async (req, res) => {
  try {
    const newPatient = new Patient(req.body);
    const savedPatient = await newPatient.save();
    console.log("👤 New Patient Saved:", savedPatient.name);
    res.status(201).json(savedPatient);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// 2) عرض كل المرضى
app.get('/api/patients', async (req, res) => {
  try {
    const patients = await Patient.find().sort({ createdAt: -1 });
    res.json(patients);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ====================================================
```

```
// Records
// ==================================================

// 3) إضافة سجل تشخيص جديد
app.post('/api/records', async (req, res) => {
  try {
    const { patientId } = req.body;

    // ✅ تأكد إن patientId موجود
    if (!patientId) {
      return res.status(400).json({ error: "patientId is required" });
    }

    // ✅ تأكد إن المريض موجود فعلاً (اختياري لكنه يحمي الداتا)
    const patient = await Patient.findById(patientId);
    if (!patient) {
      return res.status(404).json({ error: "Patient not found" });
    }

    const newRecord = new Record(req.body);
    const savedRecord = await newRecord.save();
    console.log("📝 New Diagnosis Record Saved!");
    res.status(201).json({ message: "Diagnosis Saved!", data: savedRecord });

  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// 4) عرض كل السجلات (مع بيانات المريض)
app.get('/api/records', async (req, res) => {
  try {
    const records = await Record.find()
      .populate('patientId')
      .sort({ visitDate: -1 });

    res.json(records);
  } catc
... (truncated for readability)
```